# XSD: краткая памятка

Основные конструкции XML Schema 1.0 с примерами. Серым — схема, зелёным — валидный XML, красным — невалидный.

## 1. schema

`xs:schema` — корневой элемент XSD-документа. Его непосредственные потомки называются глобальными компонентами схемы.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
</xs:schema>
```

## 2. element

`xs:element` — объявление элемента. Оно связывает имя элемента (`name`) с типом его содержимого (`type`). Любой глобально объявленный элемент может быть корнем XML-документа.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="greeting" type="xs:string"/>
</xs:schema>
```

```
<greeting>Привет</greeting>
```

```
<hello>Привет</hello>   <!-- элемент не объявлен -->
```

## 3. Встроенные типы

Встроенные типы — это типы, заданные спецификацией XSD. Они делятся на **примитивные** (`string`, `decimal`, `boolean`, `date` и др.) и **производные** от них (`integer`, `int`, `long` и др.).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="age"      type="xs:int"/>
  <xs:element name="birthday" type="xs:date"/>
</xs:schema>
```

```
<age>25</age>
```

```
<birthday>15.05.1990</birthday>   <!-- формат xs:date: ГГГГ-ММ-ДД -->
```

## 4. simpleType и restriction

**simpleType** — определение простого типа, то есть типа, значение которого представляет собой только текст, без дочерних элементов и атрибутов. Простой тип задаёт допустимые значения текстового содержимого элемента или значения атрибута. Сам `simpleType` не задаёт значений: внутри него указывается один из трёх способов вывода — `restriction`, `list` или `union`.

**restriction** — способ вывода, при котором пространство значений производного типа является подмножеством пространства значений базового типа (`base`). Условия сужения задаются фасетами.

**Взаимосвязь.** `simpleType` определяет, что создаётся (новый простой тип), а `restriction` — как (сужением базового типа). Поэтому на практике они используются вместе.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="ShortString">
    <xs:restriction base="xs:string">
      <xs:maxLength value="10"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:element name="title" type="ShortString"/>
</xs:schema>
```

```
<title>Привет</title>
```

```
<title>Очень длинный заголовок</title>   <!-- длина больше 10 -->
```

## 5. Фасеты

Фасет — условие, ограничивающее пространство значений типа.

- Разные фасеты объединяются логическим «И».
- Несколько `enumeration` или несколько `pattern` объединяются логическим «ИЛИ».
- Производный тип не может ослабить фасет базового типа.

| Фасет | Ограничивает | Применим к |
|---|---|---|
| `length`, `minLength`, `maxLength` | длину (символы; для списков — число элементов) | строкам, спискам |
| `pattern` | лексическую форму регулярным выражением (вся строка) | всем простым типам |
| `enumeration` | конечным множеством значений | всем простым типам |
| `minInclusive`, `maxInclusive`, `minExclusive`, `maxExclusive` | диапазоном (≤ / <) | числам, датам |
| `totalDigits`, `fractionDigits` | числом цифр всего / после точки | `xs:decimal` и производным |
| `whiteSpace` | нормализацией пробелов (`preserve` / `replace` / `collapse`) | всем простым типам |

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="Login">
    <xs:restriction base="xs:string">
      <xs:minLength value="3"/>
      <xs:maxLength value="10"/>
      <xs:pattern value="[a-z0-9_]+"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:simpleType name="Status">
    <xs:restriction base="xs:string">
      <xs:enumeration value="NEW"/>
      <xs:enumeration value="DONE"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:simpleType name="Money">
    <xs:restriction base="xs:decimal">
      <xs:minInclusive value="0"/>
      <xs:totalDigits value="8"/>
      <xs:fractionDigits value="2"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:element name="login"  type="Login"/>
  <xs:element name="status" type="Status"/>
  <xs:element name="sum"    type="Money"/>
</xs:schema>
```

```xml
<login>ivan_99</login>
<status>NEW</status>
<sum>123456.78</sum>
```

```xml
<login>Iv</login>       <!-- длина < 3, заглавная буква -->
<status>new</status>   <!-- регистр учитывается -->
<sum>10.555</sum>      <!-- 3 знака после точки -->
```

## 6. list и union

**list** — способ вывода, при котором значение типа является последовательностью значений типа элемента (`itemType`), разделённых пробелами.

**union** — способ вывода, при котором пространство значений типа является объединением пространств значений типов-членов (`memberTypes`).

```xml
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="IntList">
    <xs:list itemType="xs:int"/>
  </xs:simpleType>

  <xs:simpleType name="IntOrDate">
    <xs:union memberTypes="xs:int xs:date"/>
  </xs:simpleType>

  <xs:element name="numbers" type="IntList"/>
  <xs:element name="value"   type="IntOrDate"/>
</xs:schema>
```

```xml
<numbers>1 2 3</numbers>
<value>2026-10-30</value>
```

```xml
<numbers>1 два</numbers>   <!-- "два" не xs:int -->
<value>abc</value>          <!-- не xs:int и не xs:date -->
```

## 7. Именованные и анонимные типы

**Именованный (глобальный) тип** — определение с атрибутом `name`, расположенное непосредственно в `xs:schema`. На него можно ссылаться из других мест схемы и выводить из него новые типы.

**Анонимный (локальный) тип** — определение без `name`, вложенное в объявление элемента. Он относится только к этому элементу, и наследование от него невозможно.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="title">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:maxLength value="50"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

```
<title>Война и мир</title>
```

## 8. complexType и группы модели

**complexType** — определение сложного типа, то есть типа, допускающего дочерние элементы и/или атрибуты. Состав и порядок дочерних элементов задаётся **группой модели**:

- `sequence` — все элементы строго в указанном порядке;
- `choice` — ровно один из перечисленных элементов;
- `all` — все элементы в любом порядке, каждый не более одного раза. В XSD 1.0 эта группа допустима только на верхнем уровне сложного типа.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:complexType name="Person">
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:choice>
        <xs:element name="email" type="xs:string"/>
        <xs:element name="phone" type="xs:string"/>
      </xs:choice>
    </xs:sequence>
  </xs:complexType>

  <xs:element name="person" type="Person"/>
</xs:schema>
```

```
<person><name>Иван</name><email>a@b.ru</email></person>
```

```
<person><email>a@b.ru</email><name>Иван</name></person>   <!-- нарушен порядок -->
```

## 9. minOccurs, maxOccurs

`minOccurs` и `maxOccurs` — атрибуты частицы (локального элемента или группы модели), задающие допустимое число её вхождений. По умолчанию оба равны 1. Значение `unbounded` снимает ограничение сверху.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="person">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="middleName" type="xs:string" minOccurs="0"/>
        <xs:element name="phone"      type="xs:string" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<person><phone>111</phone><phone>222</phone></person>
```

```
<person><middleName>Иванович</middleName></person>   <!-- phone: 0 при minOccurs=1 -->
```

## 10. attribute

`xs:attribute` — объявление атрибута. Оно связывает имя атрибута с **простым** типом его значения.

- Атрибуты объявляются внутри `complexType`, после группы модели.
- `use="required"` делает атрибут обязательным; по умолчанию он необязателен.
- Порядок атрибутов в документе не важен.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="user">
    <xs:complexType>
      <xs:attribute name="id"   type="xs:int" use="required"/>
      <xs:attribute name="role" type="xs:string"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<user id="7" role="admin"/>
```

```
<user role="admin"/>   <!-- отсутствует обязательный id -->
```

## 11. extension

`extension` — способ вывода, при котором производный тип содержит всё содержимое базового типа и дополнительно новые атрибуты и/или элементы. Результат расширения всегда является сложным типом. Ограничения базового типа сохраняются.

- `simpleContent` — к простому значению добавляются атрибуты.
- `complexContent` — к сложному типу добавляются элементы, которые располагаются после унаследованных, а также атрибуты.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:complexType name="Price">
    <xs:simpleContent>
      <xs:extension base="xs:decimal">
        <xs:attribute name="currency" type="xs:string"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>

  <xs:complexType name="Person">
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="Employee">
    <xs:complexContent>
      <xs:extension base="Person">
        <xs:sequence>
          <xs:element name="position" type="xs:string"/>
        </xs:sequence>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>

  <xs:element name="price"    type="Price"/>
  <xs:element name="employee" type="Employee"/>
</xs:schema>
```

```
<price currency="RUB">199.99</price>
<employee><name>Иван</name><position>Разработчик</position></employee>
```

```
<employee><position>Разработчик</position><name>Иван</name></employee>
```

## 12. Итог

Сравнение способов вывода типов:

| Способ вывода | Результат | Смысл |
|---|---|---|
| `restriction` | тип того же вида | сужение: производный ⊆ базовый |
| `list` | простой тип | последовательность значений |
| `union` | простой тип | объединение типов |
| `extension` | сложный тип | дополнение базового типа |